SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	40% of CIA	**Total Marks:**	2 * 50 = 100 (2 Question)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	A. Mounish	**Reg. No.:**	19472273
Section / Batch:	CSE AI	**Date:**	08/08/2026

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.* Signature of the Student: _____

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

A. Basic API Integration and Text Generation

30. RAG-Based Research Paper Assistant: Develop a RAG application that accepts a research paper and allows users to ask questions about it. Retrieve relevant sections and generate answers.

Question 30 requires development of a RAG-Based Research Paper Assistant. The application must accept a research paper, allow users to ask questions about it, retrieve relevant sections, and generate answers.

Instructions Relevant to the Implementation

- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and suitable edge cases.
- Use readable, structured, and modular code.
- Add comments and brief documentation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Rubric

Criterion	Weightage	Level 4 Target
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow
Problem-Solving Completeness	25%	Solves the given problem completely and robustly
Code Quality	20%	Clean, well-structured, error-handled code
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases
Code Documentation	10%	Well-commented code with clear README/setup steps

2. Project Overview

The application implements a Retrieval-Augmented Generation workflow. A PDF is uploaded, its text is extracted and divided into smaller chunks. Gemini embeddings represent the document chunks and the user's query. Cosine similarity, combined with keyword matching, ranks the most relevant chunks. The top three retrieved sections are passed to Gemini to generate a grounded answer.

Main Modules

- PDF Upload and Text Extraction
- Document Chunking
- Gemini Embedding Generation
- Hybrid Retrieval using Semantic Similarity and Keyword Matching
- Top-3 Relevant Section Display
- Gemini Answer Generation
- API Error and Edge-Case Handling
- Streamlit User Interface

Technology Stack

Technology	Use
Programming Language	Python
User Interface	Streamlit
Generative AI API	Google Gemini
PDF Processing	pypdf
Numerical Processing	NumPy
Retrieval	Gemini embeddings + cosine similarity + keyword matching

3. RAG Workflow

1. Upload PDF research paper/document.

2. Extract readable text from all PDF pages.

3. Split the extracted text into overlapping chunks.

4. Generate document embeddings using Gemini.

5. Convert the user's question into a retrieval embedding.

6. Calculate semantic similarity and keyword relevance.

7. Rank chunks and retrieve the top 3 relevant sections.

8. Send the retrieved context and question to Gemini.

9. Display the generated grounded answer.

The application visibly presents the retrieved sections and similarity scores before the generated answer, satisfying the assignment's RAG demonstration requirement.

4. Setup and Execution

1. Create/open the project folder in VS Code.
2. Install the required Python packages from requirements.txt.
3. Keep the Gemini API key private. The final report does not reproduce the secret key.
4. Run the application using: streamlit run app.py
5. Open the local Streamlit address shown in the terminal.
6. Upload the PDF and enter a question.

Requirements

Recommended requirements.txt entries used by the implementation:

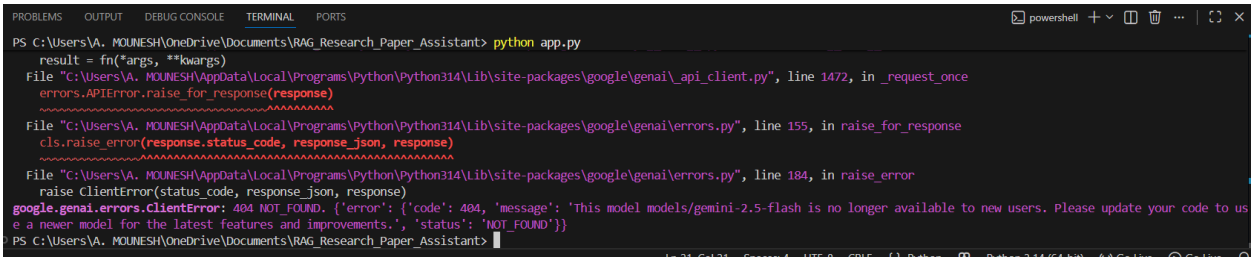
streamlit
google-genai
pypdf
numpy
python-dotenv

5. Implementation and Testing Screenshots

The following screenshots are retained from the implementation process. They are arranged in chronological order so that the document records the development, debugging, and final successful demonstrations.

Figure 1

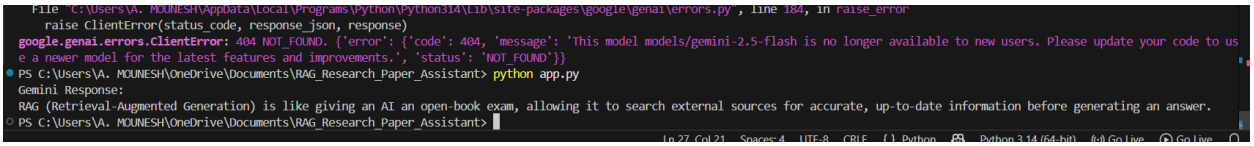
Initial Gemini API/model error shown in the VS Code terminal.



Initial Gemini API/model error shown in the VS Code terminal.

Figure 2

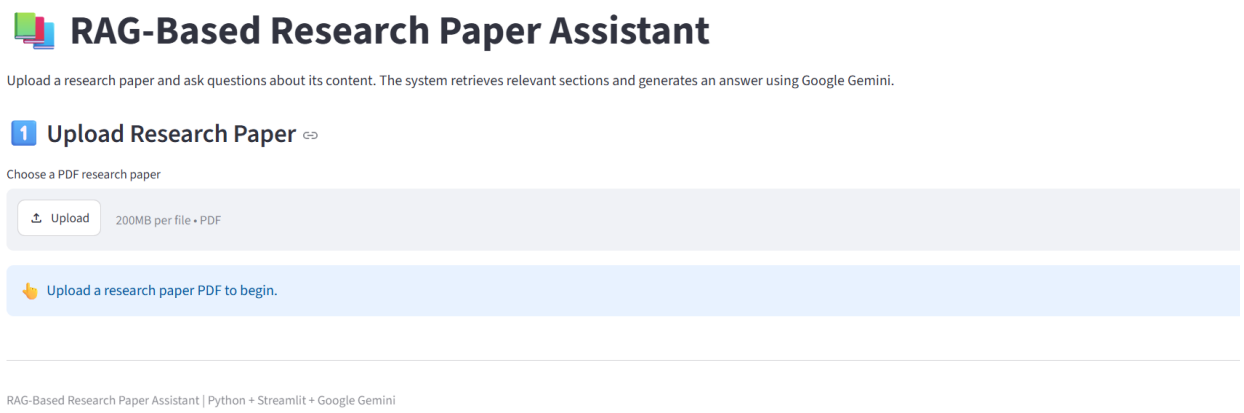
Model availability issue followed by a successful Gemini response.



Model availability issue followed by a successful Gemini response.

Figure 3

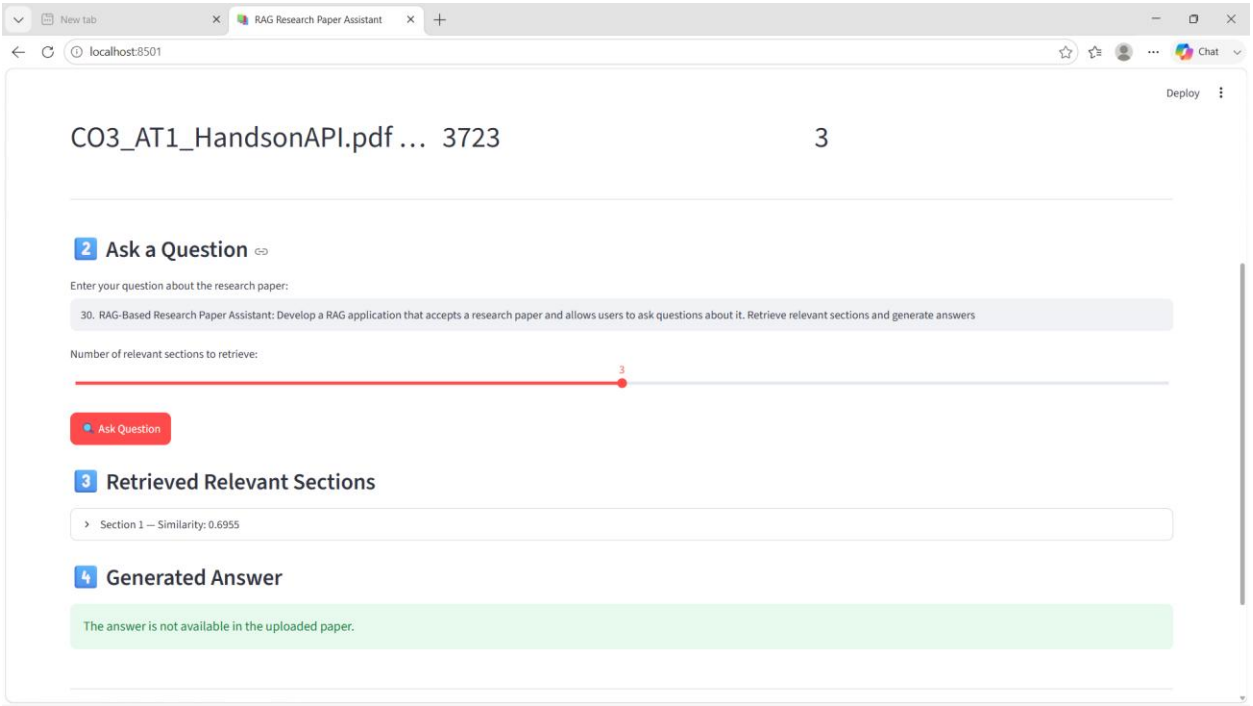
Initial Streamlit interface with PDF upload section.



Initial Streamlit interface with PDF upload section.

Figure 4

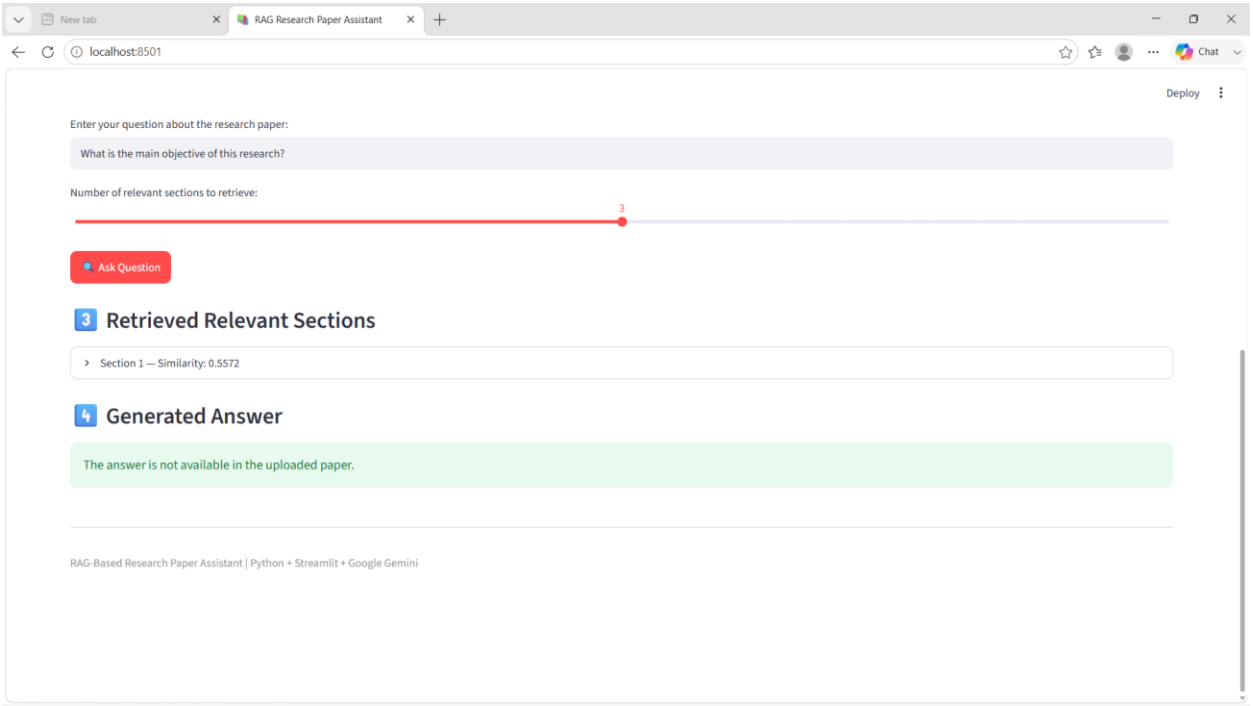
Initial RAG test where the answer was unavailable in the uploaded document.



Initial RAG test where the answer was unavailable in the uploaded document.

Figure 5

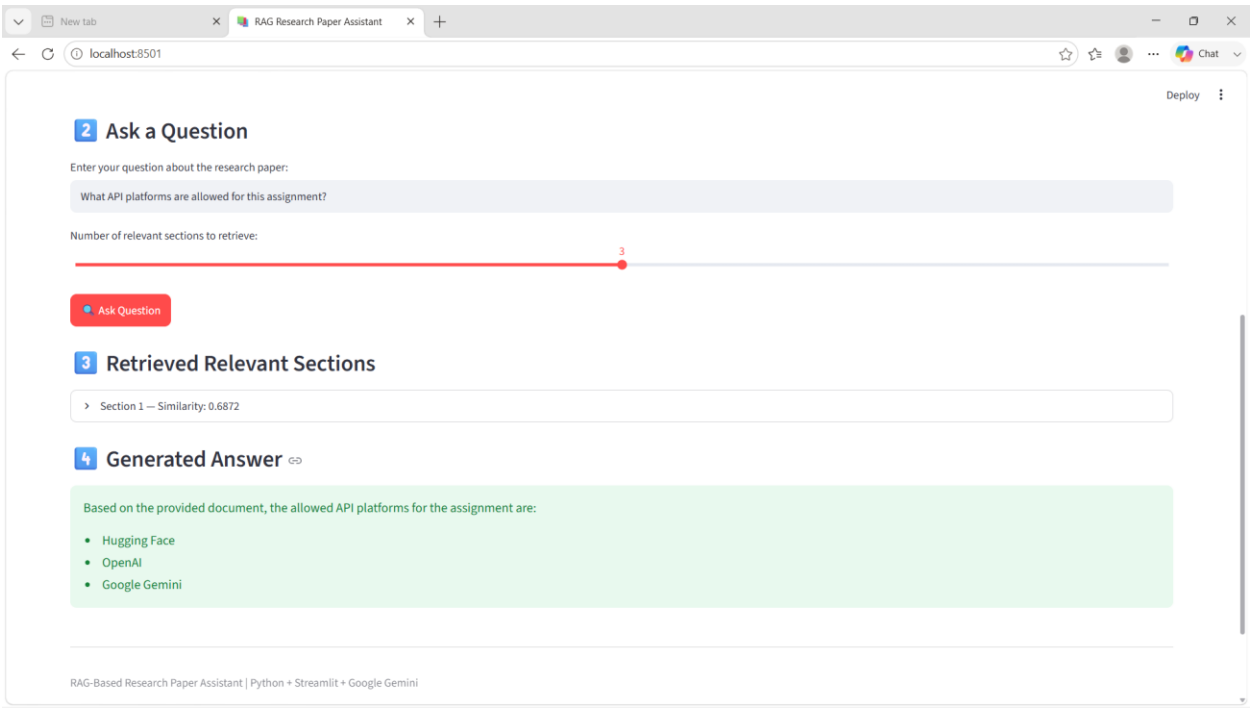
Main-objective test showing an unavailable-answer case.



Main-objective test showing an unavailable-answer case.

Figure 6

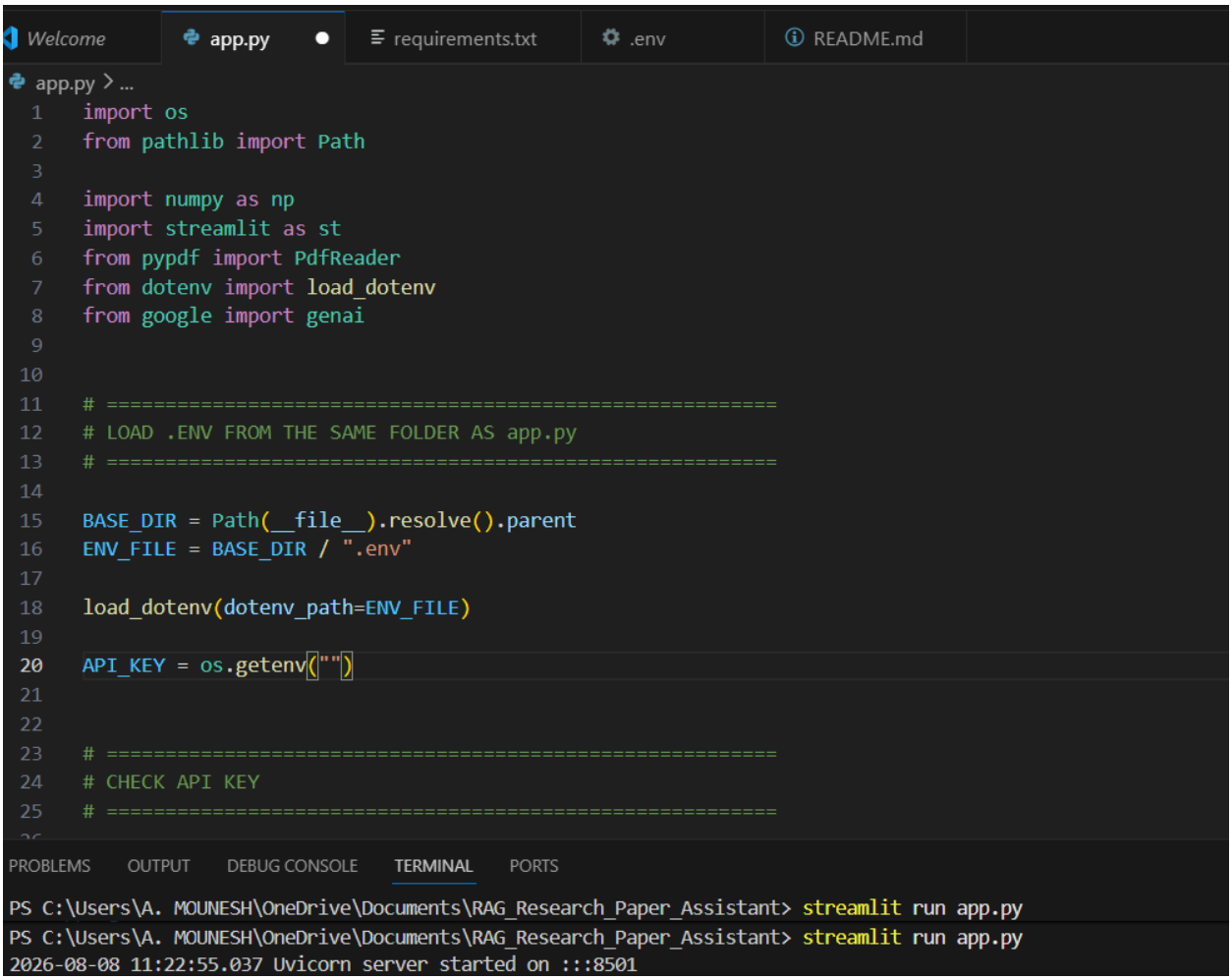
Successful API-platform test using one retrieved section.



Successful API-platform test using one retrieved section.

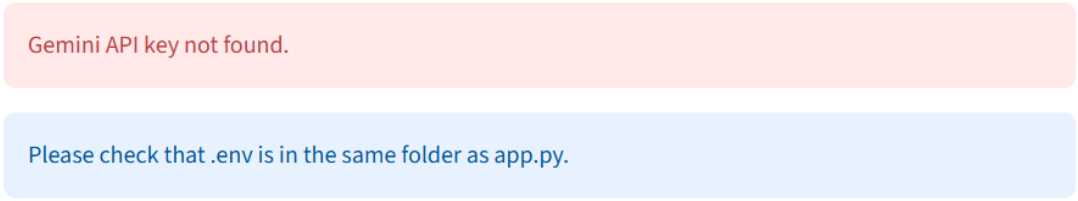
Figure 7

Early .env configuration in app.py.



Early .env configuration in app.py.

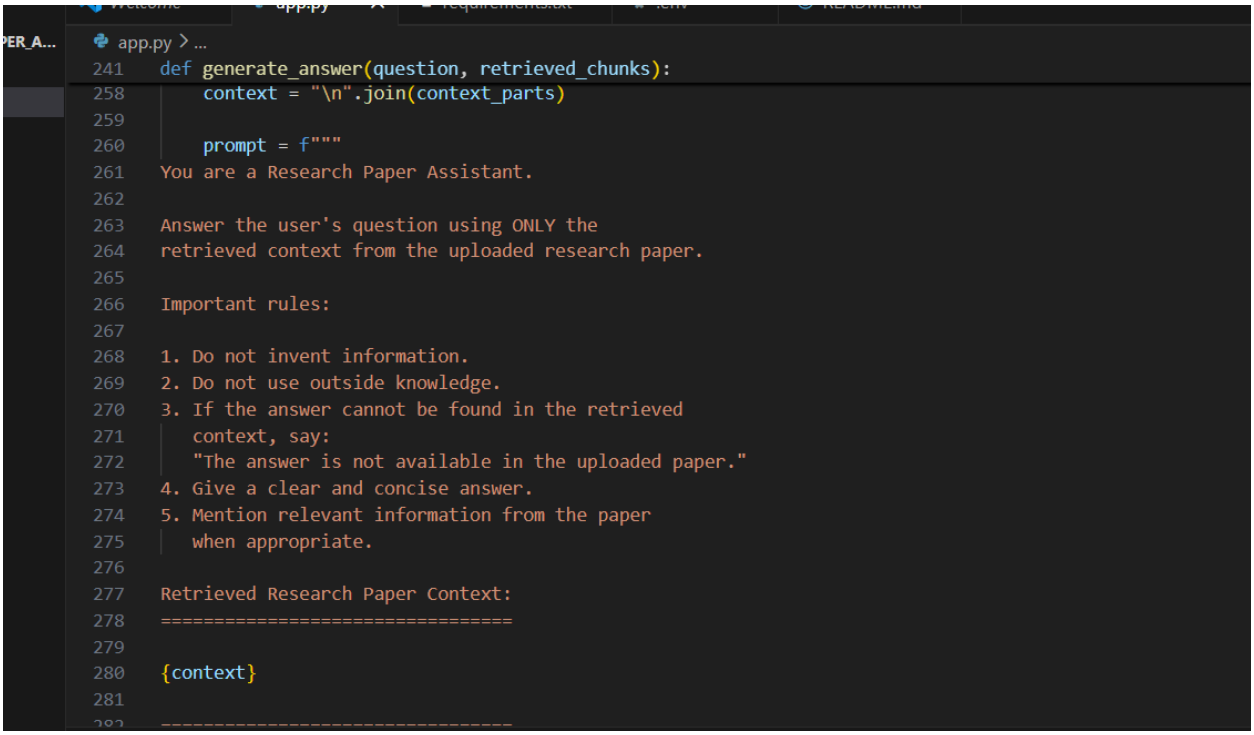
Figure 8
API-key-not-found configuration error.



API-key-not-found configuration error.

Figure 9

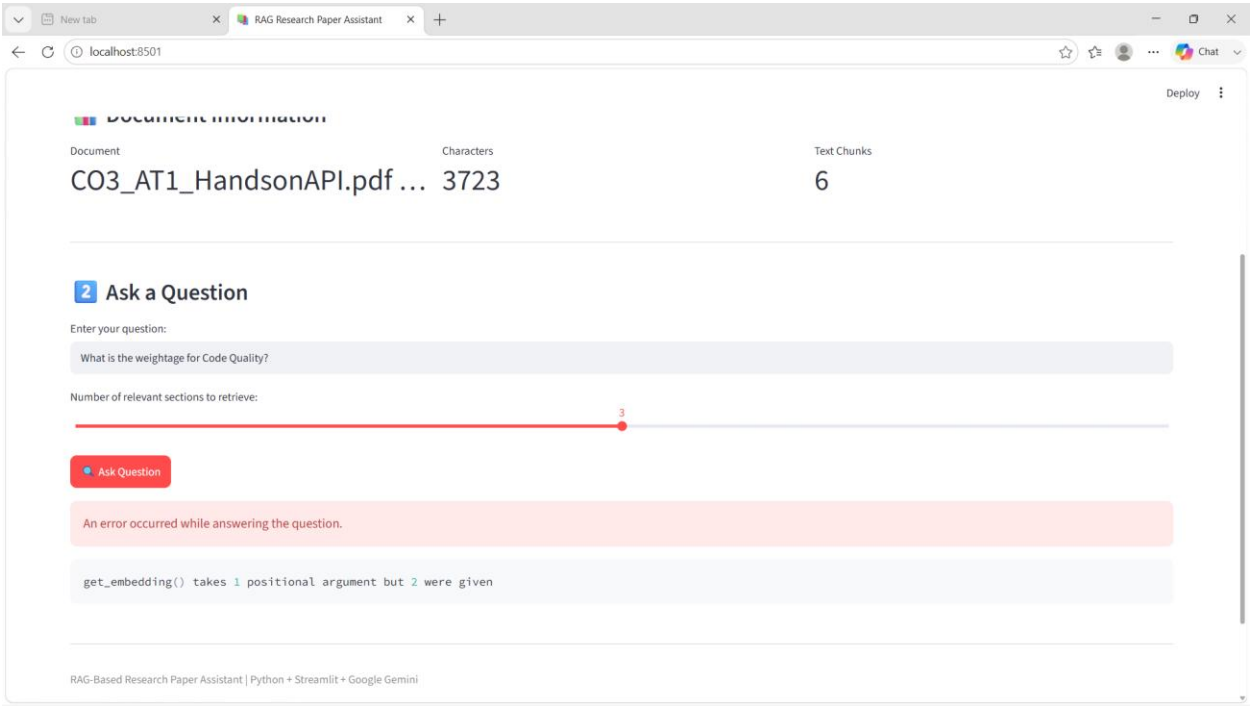
Answer-generation prompt implementation in app.py.



Answer-generation prompt implementation in app.py.

Figure 10

Embedding function argument mismatch during testing.



Embedding function argument mismatch during testing.

Figure 11

Gemini API configuration in app.py; API key redacted for security.

app.py > get_embedding

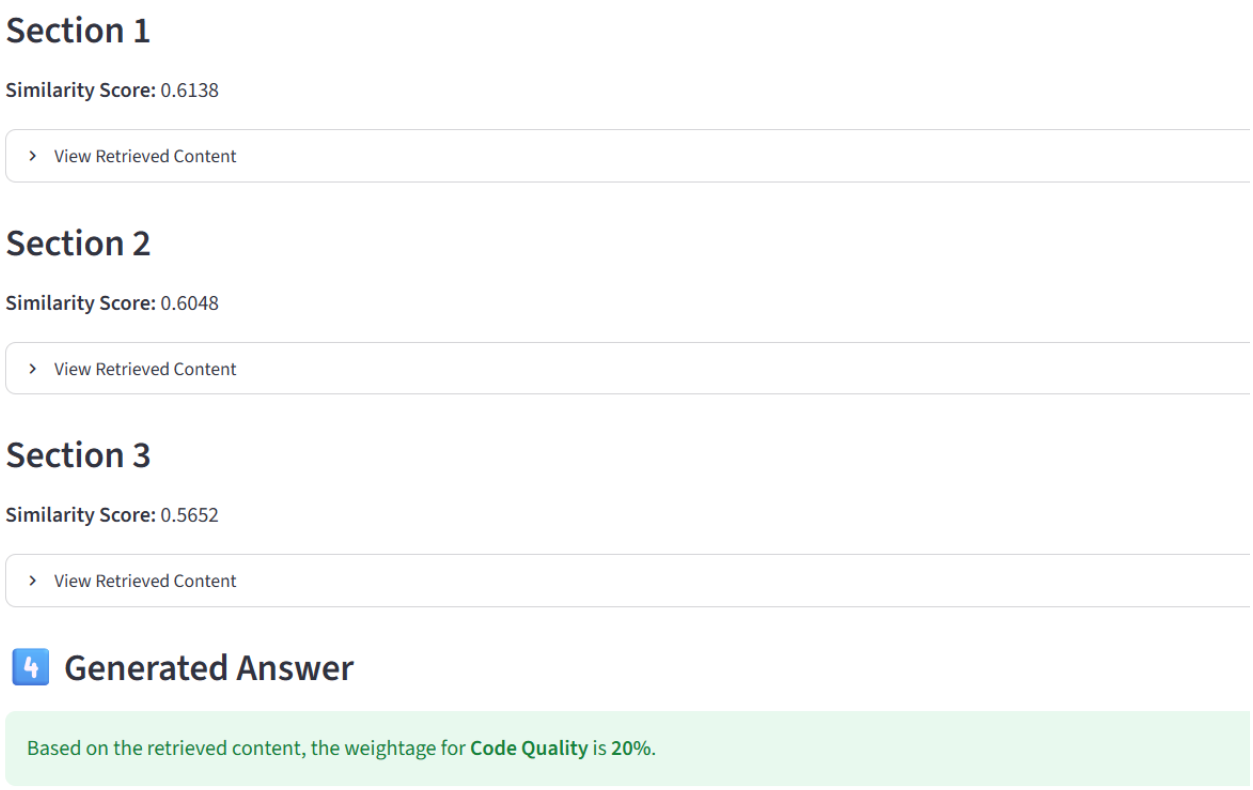
```python
from google.genai import types
import re
import time
import numpy as np
import streamlit as st
from pypdf import PdfReader
from google import genai


# ============================================================
# GEMINI API CONFIGURATION
# ============================================================

API_KEY = "AQ.Ab8RN6L

client = genai.Client(api_key=API_KEY)

GENERATION_MODEL = "gemini-3.5-flash"
EMBEDDING_MODEL = "gemini-embedding-2"


# ============================================================
```

API_KEY = "[REDACTED FOR SECURITY]"

Gemini API configuration in app.py; API key redacted for security.

Figure 12

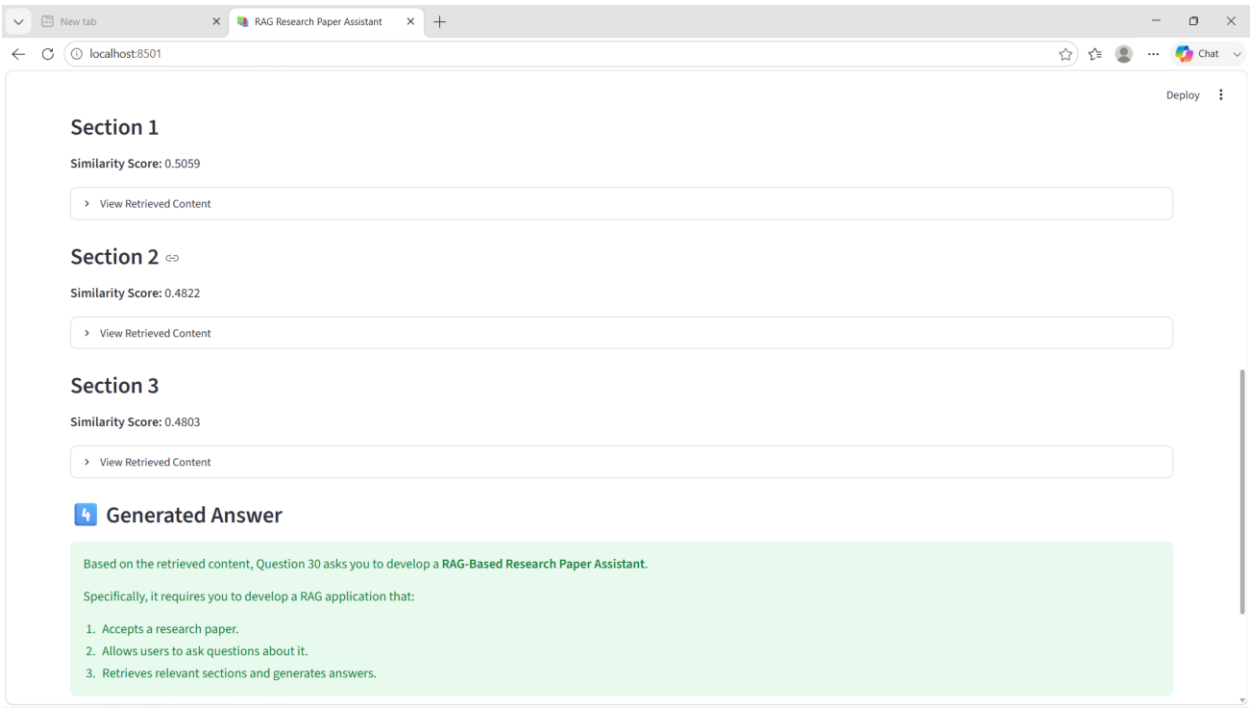
Successful Code Quality test: 20% with three retrieved sections.



Successful Code Quality test: 20% with three retrieved sections.

Figure 13

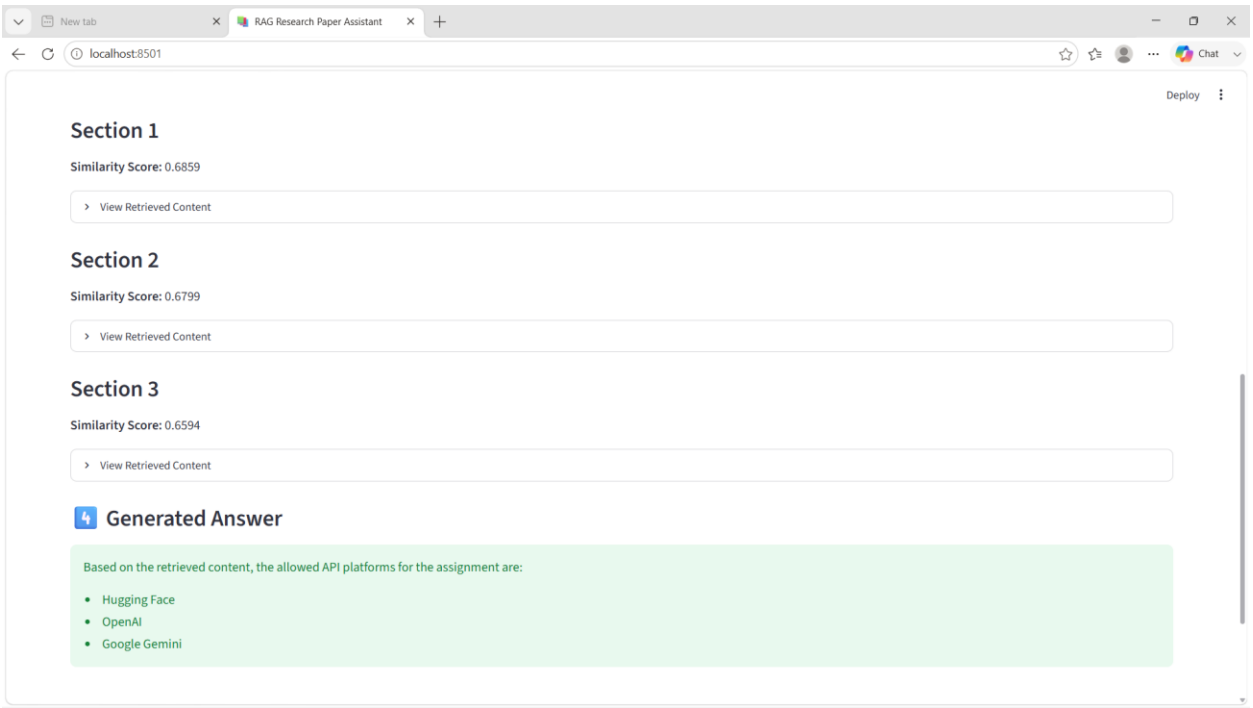
Successful Question 30 test with three retrieved sections.



Successful Question 30 test with three retrieved sections.

Figure 14

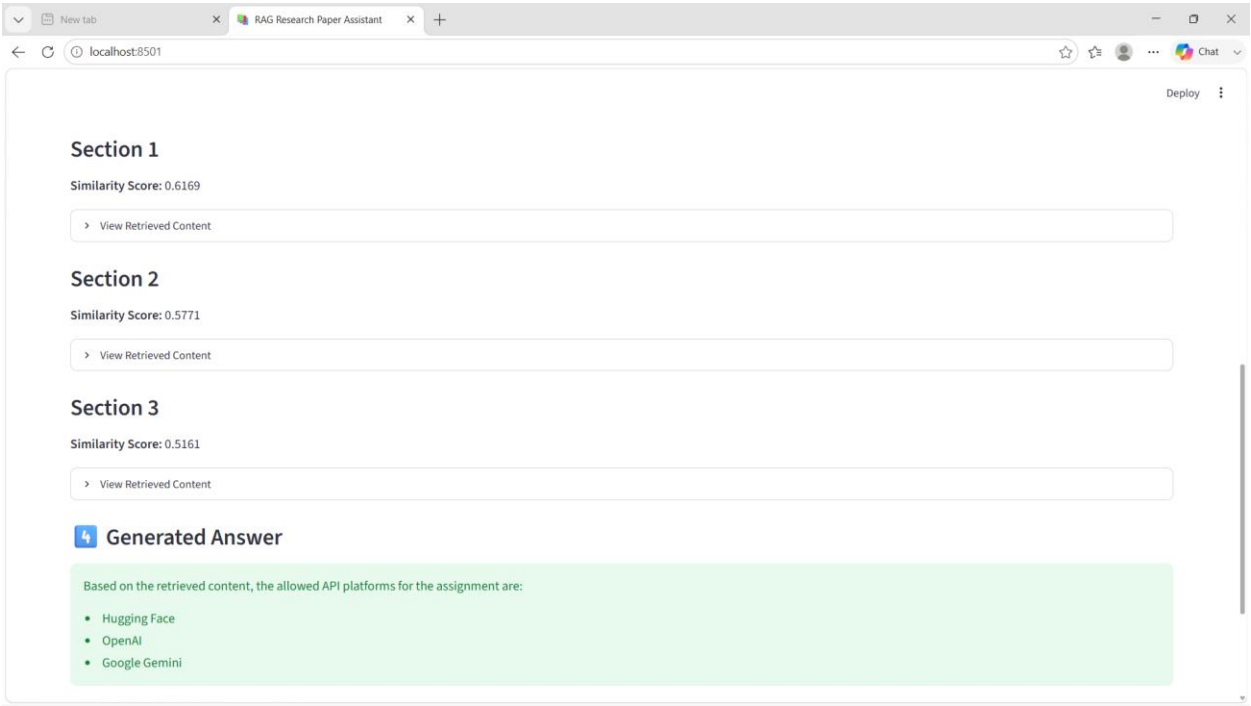
Successful API-platform test with three retrieved sections.



Successful API-platform test with three retrieved sections.

Figure 15

Repeated successful API-platform demonstration with three retrieved sections.



Repeated successful API-platform demonstration with three retrieved sections.

6. Final Demonstration Results

Test	Input	Expected/Observed Output	Status
Test Case 1	What API platforms are allowed for this assignment?	Hugging Face, OpenAI, and Google Gemini.	Successful
Test Case 2	What is the weightage for Code Quality?	20%.	Successful
Test Case 3	What does Question 30 ask us to develop?	A RAG-Based Research Paper Assistant that accepts a research paper, allows questions, retrieves relevant sections, and generates answers.	Successful
Edge Case	Question unrelated to the uploaded document	The application is designed to report that the answer is not available in the uploaded document.	Handled

The successful screenshots demonstrate three retrieved sections, similarity scores, and a generated answer. The document also records development-time API/model and configuration errors, showing that the implementation was tested and corrected.

7. Rubric Mapping

Rubric	Evidence in Implementation
API Integration Correctness – 30%	Google Gemini client, embeddings, answer generation, request/response flow, retry handling.
Problem-Solving Completeness – 25%	PDF upload, extraction, chunking, retrieval, question answering, and result display.
Code Quality – 20%	Functions separate PDF extraction, chunking, embeddings, retrieval, and generation; API errors are handled.
Input/Output Demo – 15%	Multiple successful test cases plus unavailable-answer/error cases are documented with screenshots.
Code Documentation – 10%	README/setup guidance, structured modules, comments, and this report.

8. Conclusion

The RAG-Based Research Paper Assistant was implemented using Python, Streamlit, and Google Gemini. The application accepts a PDF, extracts and chunks its content, retrieves the most relevant sections using embedding-based similarity and keyword relevance, and generates a grounded response from the retrieved context. The demonstrated test cases show successful retrieval and answer generation, while the development screenshots document API/model and configuration issues encountered and resolved.

9. Security Note

The API key shown during development was treated as a secret. The screenshot containing the key has been redacted in this Word document. API keys should not be committed to public repositories or shared in screenshots.

10. References / Source Material

Primary assignment source: CO3_AT1_HandsonAPI.pdf duplicate.pdf, provided in the conversation. The report follows the assignment's Question 30 requirements and rubric.